DevOps

Tehnička praksa uvođenja principa toka
Uspostavljanje *deployment pipeline-a*

predavanja su bazirana i islike preuzete iz knjige
“The **DevOps Handbook**: How to Create World-Class Agility, Reliability, & Security in Technology Organizations”
Gene Kim, Jez Humble, Patrick Debois, John Willis

Tehničke prakse uvođenja principe toka

- Osnovni cilj - obezbediti brz protok informacija i urađenog posla iz razvojnih ka operativnim timovima.
- Pri tome treba izbeći unošenje poremećaja kako u samu organizaciju koja razvija softver tako i kod klijenta.
- Neophodno je minimizovati rizike koji postoje kada se nove izmene primenjuju u produkcionom okruženju.
- Sve ovo postiže se implementiranjem tehničkih praksi kontinulane isporuke - *continuous delivery.*

Tehničke prakse uvođenja principe toka

- Kontinualna isporuka podrazumeva
  - **uspostavljanje *deployment pipeline-a***
  - omogućavanje brzog i pouzdanog automatizovanog testiranja
  - omogućavanje i uvođenje u praksu kontinualne integracije
  - automatizaciju i upravljanje razvojem koji omogućava isporuku uz male rizike (*low risk deployment*)

Uspostavljanje osnove *deployment pipeline*-a

- Kako bi se obezbedilo uspostavljanje brzog i pouzdanog toka (informacija i urađenog posla) od razvojnih ka operativnim timovima neophodno je obezbediti da se u svim fazama u razvojnom lancu koriste okruženja koja su ekvivalentna produkcionom okruženju.

- Potrebno je da se ova okruženja mogu kreirati automatizovano, korišćenjem skriptova i konfiguracionih informacija koje su smeštene i čuvaju se u nekom od sistema za kontrolu verzija.

- Idealno - okruženja se na osnovu ovih skriptova mogu kreirati “na zahtev”, kada su potrebna, samostalno, bez potrebe za kreiranjem “tiketa” i bez potrebe da se nešto ručno podešava.

Uspostavljanje osnove *deployment pipeline*-a

- Zašto je bitno kreiranje okruženja koja su ekvivalentna produkcionim?
  - Vrlo često se dešava da razvojni timovi nemaju nikakav osećaj niti informaciju kako se aplikacija ponaša u stvarnim, produkcionim okruženjima.
  - Prve informacije i iskustvo dobiju tek kada se aplikacija stvanro instalira na produkcioni sistem, a to je prekasno i potencijalno opasno po poslovne potrebe klijenta.

Uspostavljanje osnove *deployment pipeline*-a

- Em Campbell-Pretty (generalni direktor Enterprise Data Warehouse centra australijske telekomunikacione kompanije) je opisala svoja iskustva sa razvojem aplikacija:
  - u kompaniji je 2014 bilo 11 različitih proizvodnih tokova (projekata) u razvoju, koristili su “waterfall” pristup razvoja
  - 10 timova je stalno kasnilo u odnosu na plan, a samo jedan je na vreme isporučio aplikaciju na User Acceptance Test, ali i taj test je trajao mesecima da se otklone nedostaci - opet izazivajući kašnjenje u odnosu na plan, i krajnji rezultat nije ocenjen kao nešto što je ispunilo očekivanja klijenta
  - prešli su na agilne metode razvoja - što je popravilo, ali ne i otklonilo kašnjenja u odnosu na planove

Uspostavljanje osnove *deployment pipeline*-a

- Većina timova tokom razvoja spominjala kao najveći problem nedostatak komunikacije od strane “poslovnih struktura”

- Nakon što su anketirani šta bi trebalo promeniti da se produktivnost poboljša i pored prethodne opaske, većina timova to više nije navodila kao glavni problem već nedostatak odgovarajućih okruženja.

  - ispostavilo se da kada bi razvojnim timovima trebala okruženja na kojima bi *deployovali* i pokrenuli i testirali to što rade, morali su da kreiraju tikete IT-u (operativnim timovima), a u nekim slučajevima je trebalo i do 8 nedelja da im se na taj manuelni način obezbede potrebna okruženja

Uspostavljanje osnove *deployment pipeline*-a

- potom je formiran tim za integrisanje i bildovanje primarno sastavljen od sistem administratora i administratora baza podataka specijalizovanih u uvođenju automatizacije

- uskoro je otkriveno da se čak i do 50% koda koji je na produkciji razlikuje od onog koji je na razvojnim i testnim okruženjima

- kada je došlo do otkrivanja problema u produkciji, hotfix-ovi su primenjivani, ali nisu konzistentno *back-portovani* u sisteme za kontrolu verzija i na razvojna i testna okruženja

Uspostavljanje osnove *deployment pipeline*-a

- Timovi su onda pažljivo odradili prenos svih produkcionih izmena u sistem za kontrolu verzija i napravili automatizovane skriptove za kreiranje kontrolisanih i pouzdanih okruženja

- Vreme potrebno da se dobije odgovarajuće okruženje palo je sa 8 nedelja na 1 dan

- Upravo ova promena omogućila je da timovi konačno počnu da postižu zadane rokove, uklope se u predviđene troškove i što je jako bitno smanje broj grešaka koje su se “probile” u produkciona okruženja klijenta

Omogućavanje kreiranja “na zahtev” potrebnih okruženja (razvojnog, testnog i produkcionog)

- Iz prethodnog primera vidljivo je da je dostupnost pouzdanih okruženja koja su slična produkcionim predstavlja jedan od preduslova za isporuku kvalitetnog koda na vreme

- Važno je obezbediti da razvojni tim može na lokalnim mašinama, na zahtev kreirati okruženje slično produkcionom, kao i da u vrlo ranoj fazi mogu brzo dobiti testno okruženje

Omogućavanje kreiranja “na zahtev” potrebnih okruženja (razvojnog, testnog i produkcionog)

- Da bi se omogućilo brzo kreiranje pouzdanih okruženja sličnih produkcionim, neophodna je promena pristupa ovom problemu
  - umesto da se koraci i konfiguracije za kreiranje okruženja samo opisuju i dokumentuju treba kreirati automatizovani mehanizam koji omogućava njihovo kreiranje
  - sve neophodne detalje treba umesto u opisnu dokumentaciju uneti u skriptove koji odgovarajući alati mogu koristiti za automatizovno bildovanje okruženja
  - na ovaj način operativni timovi se oslobađaju manuelnog posla koji je sklon greškama i jako bitno - detalji pouzdane konfiguracije više nisu nešto što zna samo neki pojedinac, već je to znanje zapisano u skriptovima

Omogućavanje kreiranja “na zahtev” potrebnih okruženja (razvojnog, testnog i produkcionog)

- Moguće je automatizovati:
  - kopiranje virtuelnih okruženja (VMWare slika, pokretanje Vagrant skriptova, startovanje instance EC2 virtuelne mašine na AWS-u…)
  - izgradnju automatizovanog procesa koji startuje formiranje okruženja od “golog metala”
  - korišćenje alata za upravljanje konfiguracijama po principu “infrastruktura kao kod” (Puppet, Chef, Ansible, Salt…)
  - korišćenje alata za konfigurisanje OS-a
  - formiranje okruženja na osnovu slika virtuelnih mašina ili kontejnera
  - pokretanje novih okruženja na *cloud* okruženjima

Omogućavanje kreiranja “na zahtev” potrebnih okruženja (razvojnog, testnog i produkcionog)

- Od ovakvog pristupa koristi imaju i operativni i razvojni timovi
  - operativni timovi - jer ovakvo automatizovano kreiranje obezbeđuje ponovljivost i konzistentnost procesa kreiranja okruženja, uklanjanje mnogo ručnih koraka konfigurisanja koji često unose greške u konfiguracije
  - razvojni timovi - jer mogu brzo i na zahtev dobiti delove (ili celo) okruženje koje im je neophodno da instaliraju i testiraju razvijene komponente

Kreiranje jednog repozitorijuma kao izvora pouzdanog stanja sistema

- Osim što je potrebno obezbediti automatizovane skriptove za kreiranje okruženja neophodno je obezbediti da se na repozitorijumu sistema za kontrolu verzija nađe SVE što definiše stabilnu verziju softverskog sistema
  - dakle ne samo kod koji se razvija već i skriptove za kreiranje okruženja,
  - verzije *third-party* biblioteka koje stabilne verzije našeg softvera koriste
  - sve konfiguracione fajlove i skripte

Kreiranje jednog repozitorijuma kao izvora pouzdanog stanja sistema

- Razvojni timovi su već naviknuti da koriste sisteme za kontrolu verzija na koje *commit*-uju kod

- Ali kako je za uspešnu instalaciju neophodna korektna i testirana verzija koda, ali i dobro konfigurisano okruženje neophodno je uvesti praksu i da operativni timovi (ali i svi ostali InfoSec, QA…) sve svoje konfiguracije drže uredno ažuriranim na sistemima za kontrolu verzija

Kreiranje jednog repozitorijuma kao izvora pouzdanog stanja sistema

- Kako bi se garantovalo da i posle eventualnog katastrofalnog otkaza možemo sisteme oporaviti u kratkom vremenu, na sistemima za kontrolu verzija bi trebalo da se nalaze:
  - sav aplikativni kod i dodatni kod od kojeg je on zavisan
  - svi skriptovi za kreiranje baza podataka, statičkih i sl.
  - svi artefakti neophodni za kreiranje okruženja (VMWare ili AMI slike, Puppet ili slični skriptovi)
  - svi fajlovi za kreiranje kontejnera (Docker slike ili *compose* fajlovi)
  - svi automatizovani testovi (i opisi manuelnih testova)
  - svi skriptovi koji obezbeđuju pakovanje koda, instalacione skripte, skripte za migraciju baza podataka…
  - drugi projektni artifakti (dokumentacija korisničkih zahteva, procedure za instaliranje…)
  - svi konfiguracioni fajlovi za instaliranje na cloud platformama (AWS CloudFormation templates, Azure Stack DSCs…)

Kreiranje jednog repozitorijuma kao izvora pouzdanog stanja sistema

- Naravno nije neophodno da sve ovo bude u samo jednom repozitorijumu, moguće je kreirati za svaku grupu fajlova odgovarajući, ali je neophodno da sve ovo bude pod sistemom za kontrolu verzija

- Jako je bitno da se obezbedi da je moguće rekonstruisati ne samo aplikaciju i okruženje već i sve ono na što se oslanjala bilo koja faza razvoja (verzije kompajlera i razvojnih okruženja za slučaj da treba intervenisati u starijim verzijama)

Kreiranje jednog repozitorijuma kao izvora pouzdanog stanja sistema

- Po nekim analizama pokazalo se da na konačni uspeh projekata u zadatom roku veću ulogu ima činjenica da su operativni timovi prihvatili da sve što rade pohranjuju na sistemima za kontrolu verzija nego kada to rade samo razvojni timovi

Napraviti okruženje u kome je lakše i jeftinije rebildovati infrastrukturu nego je opravljati

- Ukoliko je kompletno obezbeđivanje infrastukture zasnovano na automatizovanim skriptama, moguće je doći u stanje kada je lakše i jeftinije iznova pokrenuti nove instance neophodne infrastrukture nego trošiti vreme na oporavljanje konfiguracija koje su otkazale

Napraviti okruženje u kome je lakše i jeftinije rebildovati infrastrukturu nego je opravljati

- Kada postoje automatizovane (ponovljive) skripte koje obezbeđuju potpuno konfigurisanu i funkcionalnu infrastrukturu, moguće je obezbediti i dodatne kapacitete kada se za to ukaže potreba (horizontano skaliranje)

- Ovakvim pristupom se izbegavaju i katastrofalne posledice otkaza infrastrukturnih komponenti koje bi bilo nemoguće iznova reprodukovati jer su u njih uloženi sati manuelnog konfigurisanja

Napraviti okruženje u kome je lakše i jeftinije rebildovati infrastrukturu nego je opravljati

- Kako bi se obezbedila konzistentnost infrastrukturnih komponenti kada god se napravi podešavanje konfiguracije, kreira određeni patch, upgrade na novu verziju, sve ove promene treba konzistentno ponoviti u svim preprodukcionim i produkcionim okruženjima.

- Mnogo je lakše ukoliko se usvoji praksa da se nijedna konfiguracija ne unosi ručno u pojedinačne sisteme, već da se isključivo ažuriraju konfiguracione skripte koje će datu izmenu dosledno primeniti na svim okruženjima.

- Dva pristupa - korišćenje alata koji automatizuju ove poslove i garantuju konzistenciju (Ansible, Puppet, Chef…), ili pristup kod koga svaka izmena kreira novu verziju fajlova za VM ili slične infrastrukturne komponente

Napraviti okruženje u kome je lakše i jeftinije rebildovati infrastrukturu nego je opravljati

- Ovaj drugi pristup predstavlja *immutable infrastructure* pattern
  - nikakve izmene na “živim” instancama infrastrukturnih komponenti se ne dozvoljavaju
  - svaka promena se prvo unosi u automatizovane skripte za bildovanje i deployment
  - kreiraju se nove instance koje preuzimaju zadatke postojećih, a stare (postojeće) se potom gase

Napraviti okruženje u kome je lakše i jeftinije rebildovati infrastrukturu nego je opravljati

- Ponekad se prihvata i praksa da se infrastrukturne komponente namerno gase i zanavljaju novim automatsi kreiranim iz skripti, baš sa ciljem da se kod svih učesnika u lancu razvoja razvije svest o tome da se ne mogu osloniti ni na kakvo specifično podešavanje sistema - sve mora raditi na komponentama koje se kreiraju iz određene verzije skriptova za dato okruženje

Promena filozofije “urađenog”

- Kada se okruženja mogu automatizovano i pouzdano kreirati na osnovu skriptova koji su pod upravljanjem sistema za kontrolu verzija, neophodno je obezbediti da razvojni timovi svakodnevno rade na takvim (production-like) okruženjima

- Na ovaj način se obezbeđuje pravovremena (a ne prekasna) potvrda da naš kod radi na odovarajućem okruženju, što omogućava da se eventualni problemi otkriju u ranim fazama

Promena filozofije “urađenog”

- Većina metodologija razvoja danas podrazumeva kratke cikluse razvoja, a zatim isporuku urađenih modula koji donose mali inkrement funkcionalnosti sistema

- Pokazalo se da ovo smanjuje i greške - što duži razvojni intervali to su rezultati prilikom isporuke i pokretanja u realnim okruženjima lošiji

- Cilj je osigurati da i razvojni timovi i QA svakodnevno rade na okruženjima koja su što sličnija realnim produkcionim

Promena filozofije “urađenog”

- Neophodna je i promena načina na koji se sagledava “urađeno”
- Pod urađenim se više ne može smatrati kod koji dokazano ispunjava funkcionalne zahteve, već se na kraju svakog razvojnog ciklusa sve mora integrisati, testirati i dokazano raditi na okruženjima koja su replika produkcionih
    - idealno bi bilo da se može postići da kod pre isporuke zaista radi i pod realnim opterećenjem i sa skupovima podataka koji se očekuju na produkciji

Promena filozofije “urađenog”

- Na ovaj način se obezbeđuje da se integrisanje, testiranje i izvršavanje koda na *production-like* sistemima obavi mnogo puta (poželjno potpuno automatizovano), pre nego što se kod uopšte i prvi puta isporuči klijentu

- Idealno, sva ova okruženja koriste i isti skup alata za nadzor i merenje performansi kao i produkciono čime se obezbeđuju brze povratne informacije